Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – III

Roll No. : T045	Name: Omkar Thukarul
Class:TYIT	Batch:1
Date of Assignment: 14-07-2026	Date/Time of Submission: 14-07-2026

Part B - Program

a) Create methods add(). multiply(), subtract() ,divide() with suitable parameters and call these methods using concept of C# delegate.

Algorithm:

- Step 1: Start.
- Step 2: Define a delegate Operation that accepts two integer parameters.
- Step 3: Create a class to calculate with methods for Addition, Multiplication, Subtraction, and Division.
- Step 4: Create an object of the calculate class.
- Step 5: Declare a delegate variable obj.
- Step 6: Assign the delegate to the Add() method and call it with two integers.
- Step 7: Assign the delegate to the Mul() method and call it with two integers.
- Step 8: Assign the delegate to the Sub() method and call it with two integers.
- Step 9: Assign the delegate to the Div() method and call it with two integers.
- Step 10: Display the results of all arithmetic operations.
- Step 11: Stop.

CODE:

```
using System;
delegate void Operation(int a, int b);
class calculate
{
   public void Add(int a ,int b)
   {
      Console.WriteLine("Addition =" + (a + b));
   }
   public void Mul(int a, int b)
   {
      Console.WriteLine("Multiplication =" + (a * b));
   }
   public void Sub(int a, int b)
   {
      Console.WriteLine("Subtraction =" + (a - b));
   }
   public void Div(int a, int b)
   {
      Console.WriteLine("Division =" + (a / b));
   }}
```

OUTPUT:

```
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2
```

```
public class class1
{
   static void Main()
   {
      calculate c = new calculate();
      Operation obj;

      obj = c.Add;
      obj(10,20);
      obj = c.Mul;
      obj(4, 8);
      obj = c.Sub;
      obj(5, 8);
      obj = c.Div;
      obj(25, 5);
    }
}
```

b) Write a program using multicast delegates.

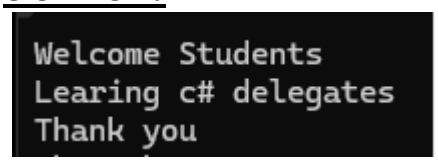
Algorithm:

- Step 1: Start.
- Step 2: Define a delegate Message with no parameters.
- Step 3: Create a class Mesg with methods Wel(), Hello(), and Th() to display messages.
- Step 4: Create an object of the Mesg class.
- Step 5: Declare a delegate variable obj.
- Step 6: Assign the Wel() method to the delegate.
- Step 7: Add the Hello() and Th() methods to the delegate using the += operator.
- Step 8: Invoke the delegate to execute all three methods in sequence.
- Step 9: Stop.

CODE:

```
using System;
delegate void Message();
class Mesg
{
   public void Wel()
   {
      Console.WriteLine("Welcome Students");
   }
   public void Hello()
   {
      Console.WriteLine("Learing c# delegates");
   }
   public void Th()
   {
      Console.WriteLine("Thank you");
   }
}
```

OUTPUT:



```
public class Program
{
   static void Main(string[] args)
   {
      Mesg m = new Mesg();
      Message obj;

      obj= m.Wel;
      obj += m.Hello;
      obj += m.Th;
      obj();
      Console.WriteLine("Bhavesh T039");
   }
}
```

c) Create a class BankAccount with AccountNumber and Balance. Implement property for Balance with validation (Balance cannot be negative).

Algorithm:

- Step 1: Start.
- Step 2: Create a class Bankacc with properties Accnum and Balance.
- Step 3: Use a private variable balance to store the account balance.
- Step 4: Validate the Balance property so that only non-negative values are accepted.
- Step 5: Create an object of the Bankacc class.
- Step 6: Assign an account number and a valid balance.
- Step 7: Display the account number and balance.
- Step 8: Assign a negative balance value.
- Step 9: Display an error message if the balance is negative.
- Step 10: Stop.

CODE:

```
using System;
class Bankacc
{
   public int Accnum { get; set;  }
   private double balance;
   public double Balance
   {       get { return balance; }
      set { if (value >= 0)
            balance = value;
         else
            Console.WriteLine("Balance cannot be negative");
      }
   }
}
class Program
{
   static void Main()
   {
      Bankacc b = new Bankacc();
```

```
            b.Accnum = 100;
            b.Balance = 5000;
            Console.WriteLine("Account number:"+b.Accnum);
            Console.WriteLine("Balance: "+b.Balance);
            b.Balance = -1000;
        }
    }
```

OUTPUT:

```
Account number:100
Balance: 5000
Balance cannot be negative
```